

Competitive Programming

Section 27: Bitmask DP

Competitive Programming Course

2026-07-10

Table of contents

1	Why bitmask DP matters	3
2	Vocabulary and bit operations	4
3	Recognizing bitmask DP	5
4	Pattern 1: coverage DP	6
5	Pattern 2: assignment DP	8
6	Pattern 3: subset plus last	9
7	Iteration order	10
8	Complexity intuition	11
9	C++ implementation notes	12
10	Python implementation notes	13
11	Proof habits	14
12	Common mistakes	15
13	Local judge workflow	16
14	Submission checklist	17
15	Exercises	18

1 Why bitmask DP matters

Many contest problems ask you to make decisions about a small set:

```
Which skills are already covered?  
Which jobs have already been used?  
Which cities have already been visited?  
Which special vertices have already been collected?
```

If the number of items is large, a set state is impossible. But when the statement says $n \leq 16$, $n \leq 18$, or $n \leq 20$, the intended solution is often to store the set as an integer.

This is **bitmask DP**: dynamic programming where a bit pattern represents a subset.

The key contest skill is not memorizing a formula. It is deciding:

```
What does this mask mean?  
What extra information, if any, must be stored besides the mask?  
How do I add one item while preserving the meaning of the state?
```

2 Vocabulary and bit operations

For n items, a mask is an integer from 0 to $2^n - 1$.

Bit i is 1 if item i is present and 0 otherwise.

For four items:

```
mask = 13
binary = 1101

item:   3 2 1 0
bit:    1 1 0 1
set:    {0, 2, 3}
```

Common operations:

Operation	C++	Python	Meaning
full mask	$(1 \ll n) - 1$	$(1 \ll n) - 1$	all items present
test bit i	$(\text{mask} \gg i) \& 1$	$(\text{mask} \gg i) \& 1$	is item i present?
add item i	$\text{mask} \mid (1 \ll i)$	$\text{mask} \mid (1 \ll i)$	force bit i to 1
remove item i	$\text{mask} \& \sim(1 \ll i)$	$\text{mask} \& \sim(1 \ll i)$	force bit i to 0
union masks	$a \mid b$	$a \mid b$	items in either set
intersection	$a \& b$	$a \& b$	items in both sets
count bits	<code>__builtin_popcount(mask)</code>	<code>mask.bit_count()</code>	subset size

In prose, we usually number items from 0 internally even if input is 1-indexed.

3 Recognizing bitmask DP

Bitmask DP is plausible when:

Constraint or shape	Typical interpretation
$n \leq 20$	one-dimensional DP over all subsets
$n \leq 18$ and assignment/permutation	<code>dp[mask]</code> , next row from <code>popcount(mask)</code>
$n \leq 16$ and route/order matters	<code>dp[mask][last]</code>
all subsets of a small set appear	enumerate masks from 0 to $2^n - 1$
every object is used once	mask marks used objects
shortest route visits special nodes	mask marks visited special nodes, plus current node

It is often wrong when $n = 10^5$. A 2^n table is only for small n .

4 Pattern 1: coverage DP

Suppose each candidate covers some skills and has a cost. We want minimum cost to cover all skills.

State:

```
dp[mask] = minimum cost after considering some candidates
           such that the covered skill set is mask
```

Transition for a candidate with cost c and skill mask offer:

```
new_mask = mask | offer
dp2[new_mask] = min(dp2[new_mask], dp[mask] + c)
```

We copy the old table first:

```
dp2 = dp
```

That represents not choosing the candidate. Updating $dp2$ represents choosing the candidate.

Why not update dp in place? For this particular minimum-cost choose-any problem, in-place can be made to work because choosing the same candidate twice does not improve a positive-cost answer. But the safer habit for “consider each item once” DP is to use a fresh table. It keeps the meaning clear: after candidate i , each candidate among the first i has been considered at most once.

Worked trace:

```
n = 3 skills
full = 111

Candidate A: cost 5, skills {0,1} -> offer 011
Candidate B: cost 4, skills {2}   -> offer 100
Candidate C: cost 7, skills {0,2} -> offer 101
```

Start:

```
dp[000] = 0
all others = INF
```

After A:

```
dp[000] = 0
dp[011] = 5
```

After B:

```
from 000 choose B -> 100 cost 4  
from 011 choose B -> 111 cost 9
```

After C, full coverage remains at cost 9.

5 Pattern 2: assignment DP

There are n workers and n jobs. Each worker must get one unused job.

A direct state might seem to need both worker index and job set:

```
dp[i][mask] = best after assigning first i workers and using jobs in mask
```

But i is already determined by the mask:

```
i = number of used jobs = popcount(mask)
```

So the state can be one-dimensional:

```
dp[mask] = best profit after assigning the first popcount(mask) workers  
           to exactly the jobs in mask
```

Transition:

```
i = popcount(mask)  
for each unused job j:  
    next = mask | (1 << j)  
    dp[next] = max(dp[next], dp[mask] + profit[i][j])
```

Small trace with two workers:

```
profit =  
worker 0: job0=3 job1=8  
worker 1: job0=6 job1=4
```

States:

```
dp[00] = 0  
  
mask 00, worker 0:  
    choose job 0 -> dp[01] = 3  
    choose job 1 -> dp[10] = 8  
  
mask 01, worker 1:  
    choose job 1 -> dp[11] = 7  
  
mask 10, worker 1:  
    choose job 0 -> dp[11] = max(7, 14)
```

Answer: 14.

6 Pattern 3: subset plus last

If the order matters, the mask alone is not enough.

For a route problem:

```
dp[mask] = best cost to have visited mask
```

does not tell us where we are standing, so we do not know the cost of the next move. We need:

```
dp[mask][last] = minimum cost to start at 0,  
                  visit exactly the vertices in mask,  
                  and finish at last
```

Initial state:

```
dp[001...0][0] = 0
```

Transition:

```
for each unvisited nxt:  
    dp[mask | (1 << nxt)][nxt]  
    = min(dp[...], dp[mask][last] + cost[last][nxt])
```

Final answer for a cycle returning to start:

```
min over last: dp[full][last] + cost[last][0]
```

Diagram:

```
state: mask = {0, 2, 4}, last = 2
```

```
visited:  0 ---- 4  
          \    |  
           \   |  
            2 <- current location
```

```
unvisited choices can be appended one at a time
```

7 Iteration order

Most beginner bitmask DP tables can be filled by increasing mask:

```
for (int mask = 0; mask < (1 << n); mask++) {  
    ...  
}
```

This works when transitions only add bits. Adding a bit always moves to a larger mask, so a state never depends on a future state.

For `dp[mask][last]`, use the same outer mask loop, then loop over `last`.

8 Complexity intuition

There are 2^n masks.

DP shape	States	Transition factor	Typical complexity
coverage	2^n	candidates	$O(m \cdot 2^n)$
assignment	2^n	unused jobs	$O(n \cdot 2^n)$
route/TSP	$n \cdot 2^n$	next vertex	$O(n^2 \cdot 2^n)$

Rough limits in C++:

```
2^20 ~= 1,048,576      usually fine
20 * 2^20 ~= 21 million
20^2 * 2^20 ~= 419 million, often too high
16^2 * 2^16 ~= 16 million, fine
```

Python is slower, but these local problems keep sizes within reasonable bounds for clean Python.

9 C++ implementation notes

Use `long long` for costs and profits.

```
const long long INF = (1LL << 62);  
vector<long long> dp(1 << n, INF);
```

For bit counts:

```
int k = __builtin_popcount((unsigned)mask);
```

When `n` can be 20 or more, write `1 << n` only if `n` fits in 31 bits. In this course section, `n` $\leq$ 20, so `int` masks are fine.

10 Python implementation notes

Python has arbitrary precision integers, so bit shifts are convenient:

```
full = (1 << n) - 1
if (mask >> i) & 1:
    ...
```

Use `mask.bit_count()` for popcount. It is implemented efficiently.

For large DP lists, avoid dictionaries unless the reachable state set is truly sparse. Lists are faster:

```
dp = [10**30] * (1 << n)
```

11 Proof habits

Bitmask DP proofs usually follow induction on subset size.

Template:

```
Base: the empty or start mask has the correct value.  
Induction step: assume all states with k chosen items are correct.  
Every state with k+1 chosen items is reached by adding one valid item to a  
correct k-item state, and the transition takes the best among all such last  
choices.  
Therefore the new state is correct.
```

For route DP, the proof must mention why `last` is enough. Once the visited set and current vertex are known, the future cost does not depend on the exact previous order except through the accumulated optimal cost stored in the DP.

12 Common mistakes

```
Using input skill numbers directly without subtracting 1.  
Forgetting that full mask is  $(1 \ll n) - 1$ , not  $1 \ll n$ .  
Using dp[mask] for route problems where current position matters.  
Using popcount(mask) after adding the next job instead of before.  
Returning dp[full] in TSP without adding the edge back to the start.  
Allocating  $O(n \cdot 2^n)$  memory when  $O(2^n)$  is enough.  
Using int for answers that can exceed  $2^{31} - 1$ .
```

13 Local judge workflow

Run only this section:

```
CP_TARGET=solution python3 sections/27_bitmask_dp/check.py
```

When solving yourself, omit `CP_TARGET=solution`:

```
python3 sections/27_bitmask_dp/check.py
```

The randomized tests include edge cases such as one item, asymmetric route costs, and impossible coverage.

14 Submission checklist

Before submitting a bitmask DP solution:

1. Write down what each bit means.
2. Write down what `dp[mask]` or `dp[mask][last]` means.
3. Confirm the start state.
4. Confirm transitions only use valid missing/present bits.
5. Confirm the answer state: full mask, maybe plus final return edge.
6. Estimate states times transition factor.
7. Use long long / Python int for large sums.

15 Exercises

Use *A* to make the covered-skill mask explicit, *B* to match the number of assigned workers to the mask popcount, and *C* to state the start and end of the route. Test mask zero, the full mask, and one missing transition.